

ICHIGO CHESS

Replacing the Node.js / Express Backend with Python / FastAPI

A comprehensive HLD and LLD migration guide with code, file structures, runtime explanations, transferable concepts, and critical review

Existing backend	Node.js + Express + better-sqlite3 + chess.js
Recommended backend	Python + FastAPI + Uvicorn + sqlite3/SQLAlchemy + python-chess
Frontend	Retained: HTML + CSS + browser JavaScript modules
Primary migration strategy	Contract-first, endpoint-by-endpoint replacement
Prepared	25 July 2026

Technical basis: the supplied Ichigo Chess architecture report, plus current official FastAPI, Express, Python, Uvicorn, and python-chess documentation reviewed for this guide.

Document scope, evidence, and how to read this guide

This document answers a practical architectural question: how can the existing Ichigo Chess backend be rewritten in Python without losing the browser experience, API behavior, chess correctness, persistence, engine integration, or puzzle pipeline? It starts with the system boundary, then moves down through runtime mechanics, FastAPI concepts, file organization, complete endpoint slices, testing, deployment, migration strategy, and a critical design review.

Core conclusion: Python can replace nearly all server-side Node.js code. It does not normally replace the browser-side JavaScript that handles clicks, drag-and-drop, DOM updates, clocks, highlights, and fetch calls. The browser/server HTTP contract is the seam that makes the replacement possible.

Evidence labels used in this document

Label	Meaning
Preserved	A behavior or boundary that should remain unchanged during migration.
Python replacement	A direct Python/FastAPI implementation of a current Node/Express responsibility.
Recommended	An architectural improvement suitable for the Python rewrite, not proof of current implementation.
Caution	A place where a line-by-line translation is misleading or operationally unsafe.

Assumptions

- The frontend continues to be served from the same origin whenever practical.
- The frontend continues to use the current endpoint names and JSON field names during migration.
- The server remains authoritative for legal Play moves, terminal results, persisted history, and PGN.
- Python examples target modern Python (3.12 or later) and Pydantic v2-style models.
- SQLite remains the initial persistence technology; SQLAlchemy is discussed as an optional next step.
- Stockfish or another UCI engine remains an external process rather than Python code executed in-process.

Executive summary

The correct migration is not “translate every JavaScript line into Python.” The correct migration is “preserve externally visible contracts and re-implement each backend responsibility using Python-native framework and runtime patterns.” HTTP methods, URLs, JSON DTOs, layering, controllers, repositories, transactions, adapters, and source-of-truth boundaries transfer well. Express-specific request/response mutation, middleware chaining with next(), Node module semantics, npm scripts, and the exact event-loop/process model do not transfer literally.

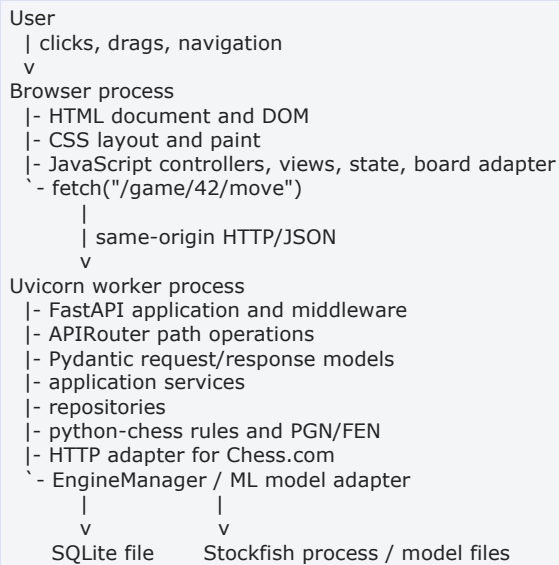
Decision area	Recommended choice	Reason
Web framework	FastAPI	Strong request/response models, dependency injection, automatic OpenAPI, and natural fit with ML/chess Python libraries.
ASGI server	Uvicorn	Listens on the TCP port and runs the FastAPI ASGI application.
Chess rules	python-chess	Legal move generation, FEN, PGN, outcomes, and UCI engine integration.
Database	sqlite3 first; SQLAlchemy later	Raw sqlite3 is close to the existing design and easy to reason about; ORM adoption should be deliberate.
External HTTP	httpx.AsyncClient	Async Chess.com/API requests without blocking the event loop.
Engine ownership	Lifespan-managed EngineManager	Start once per process, serialize engine commands, close cleanly.

Decision area	Recommended choice	Reason
Long analysis	Durable job queue when needed	FastAPI BackgroundTasks are not a durable distributed job system.
Migration	Endpoint-by-endpoint contract replacement	Limits risk and keeps the frontend operational throughout the rewrite.

Most important operational difference: A Uvicorn worker is a separate operating-system process with separate memory. If every worker loads a model and launches a Stockfish process, memory use multiplies. This is a deployment architecture decision, not merely a command-line option.

Recommended final topology

Architecture diagram: browser retained, backend replaced



Reading path

1. Read Parts I-II for the big-picture runtime and Node-to-FastAPI concept transfer.
2. Read Parts III-IV for the recommended project structure and full code slices.
3. Read Part V before writing async endpoints, engine integrations, or long analysis jobs.
4. Read Parts VI-VII for testing, migration, deployment, and alternatives.
5. Read Part VIII for the critical review and final recommendation.

Contents and navigation map

Section	Purpose
Part I	The big picture: what Python replaces and what remains in the browser
Part II	Node.js/Express concepts mapped to Python/FastAPI
Part III	Recommended Python architecture and file structures
Part IV	End-to-end implementation slices with code
Part V	Concurrency, engines, ML, long-running work, and process boundaries

Section	Purpose
Part VI	Testing, observability, security, and deployment
Part VII	Incremental migration plan and framework alternatives
Part VIII	Critical review, risks, corrections, and final recommendation
Appendix A	Node-to-FastAPI cheat sheet
Appendix B	Complete sample project skeleton
Appendix C	Glossary and source references

Part I - The big picture

1. The migration boundary: replace the server, not the web

Ichigo is a distributed application even when every component runs on one laptop. The browser and server are separate runtimes with separate memory. They communicate through HTTP. That means the browser does not import server functions and the server does not directly manipulate browser DOM elements. The browser sends a request; the server returns a response. Because this boundary is language-neutral, the server can be rewritten in Python while the browser code remains operational.

1.1 What Python can replace

- Express application creation, route registration, middleware, and error handling.
- Game creation, move validation, game reconstruction, terminal-state detection, FEN, and PGN generation.
- SQLite reads, writes, and transactions.
- Opening-book lookup and UCI engine coordination.
- Chess.com HTTP calls and response normalization.
- Puzzle extraction, engine feature calculation, ML scoring, and ranking.
- Server startup and shutdown resource management.
- Server-side tests, logging, configuration, authentication, and deployment logic.

1.2 What normally remains JavaScript

- DOM event listeners for buttons, forms, square clicks, drag start, drop, and resize.
- Rendering move lists, clocks, banners, logs, progress, and highlights without page reloads.
- Calling `fetch()` and reacting to resolved or rejected browser promises.
- Interacting with `chessboard.js` or another browser board library.
- Browser-only APIs such as `document`, `window`, `localStorage`, `AbortController`, and CSS class changes.

Do not confuse Python templates with browser Python: A Python server can render HTML templates before sending a page, but once the page is in the browser, interactive behavior still normally uses JavaScript. Technologies that run Python in WebAssembly exist, but they are not the recommended foundation for this interactive chess application.

2. The three runtimes you must keep separate

Runtime	Owns	Does not own
Browser	DOM, pixels, input events, local UI state, fetch requests	SQLite connection, Stockfish process, server secrets
FastAPI/Uvicorn worker	HTTP routes, validation, application workflows, repository calls	Browser DOM or board pixels
Engine/worker process	Chess search or long analysis execution	HTTP routing or browser rendering

2.1 Same-origin remains the simplest deployment

Same-origin resolution

```
Page URL:      http://localhost:8080/
Browser fetch: /game/42/move
Resolved URL:  http://localhost:8080/game/42/move
Server:       Uvicorn listening on port 8080
CORS required: No, because scheme + host + port are the same
```

If the frontend is served from a separate development server such as `http://localhost:5173` and the API runs at `http://localhost:8080`, the origins differ and CORS configuration becomes necessary. This is a browser security rule, not a FastAPI-specific requirement.

3. What starts when the Python backend runs

3.1 Node/Express startup

```
# package.json
{
  "scripts": {
    "dev": "PORT=8080 node server.js"
  }
}

# terminal
npm run dev
```

npm dispatches the script, Node evaluates server.js, Express routes are registered, and app.listen creates the HTTP listener. Route registration does not create one server per route; one listener dispatches requests through the Express stack.

3.2 Python/FastAPI startup

```
# terminal: explicit Uvicorn command
uvicorn app.main:app --host 0.0.0.0 --port 8080 --reload

# production-style command without reload
uvicorn app.main:app --host 0.0.0.0 --port 8080
```

Uvicorn is the server program that owns the socket. It imports app.main, retrieves the FastAPI object named app, and runs it as an ASGI application. FastAPI defines routing, validation, middleware, and application behavior; Uvicorn accepts network connections and drives the ASGI lifecycle.

The smallest useful FastAPI application

```
# app/main.py
from fastapi import FastAPI

app = FastAPI(title="Ichigo Chess API")

@app.get("/health")
def health() -> dict[str, str]:
    return {"status": "ok"}
```

3.3 Import-time work versus lifespan work

Python executes module top-level statements during import. Starting Stockfish, opening network clients, or loading large models directly at module import makes testing, reloads, and worker startup difficult. FastAPI lifespan is the clearer boundary for resources that must start before requests and close after the process stops.

Application lifespan: startup before yield, shutdown after yield

```
from contextlib import asynccontextmanager

from fastapi import FastAPI

@asynccontextmanager
async def lifespan(app: FastAPI):
    app.state.resources = await build_resources()
    try:
        yield
    finally:
        await app.state.resources.close()

app = FastAPI(lifespan=lifespan)
```

4. End-to-end request flow after migration

1. The user clicks a square or drops a chess piece in the browser.
2. The JavaScript controller creates a UCI move and calls the existing API gateway.
3. fetch sends POST /game/42/move with a JSON body.
4. The operating system delivers bytes to Uvicorn on port 8080.

5. Uvicorn creates an ASGI request scope and invokes the FastAPI application.
6. FastAPI matches the method and path, resolves dependencies, validates path/body values, and calls the path operation function.
7. The route delegates to GameService, which reconstructs the board through python-chess and calls the repository.
8. The repository commits the accepted move to SQLite.
9. The service returns a typed outcome; FastAPI serializes the response model as JSON.
10. The browser receives the response, reconciles its projected board to the authoritative FEN/history, and renders.

Sequence summary

Browser JS	FastAPI backend
-----	-----
handleDrop("e2", "e4")	
-> PlayApi.submitMove()	
-> fetch POST /game/42/move	-> Uvicorn socket
	-> middleware
	-> route validation
	-> GameService.submit_move()
	-> python-chess + SQLite
<- response.json()	<- MoveResponse JSON
-> reconcile state	
-> render DOM and board	

Part II - Node.js/Express concepts mapped to Python/FastAPI

5. Transferability classification

Category	Examples	Migration meaning
Directly transferable concept	HTTP routes, JSON DTOs, controllers, services, repositories, adapters, transactions	Keep the design idea; rewrite syntax and framework integration.
Transferable with different mechanism	Middleware, dependency injection, async I/O, lifecycle, error propagation	Preserve intent, but use FastAPI/ASGI-native features.
Not directly transferable	req/res mutation, next(), npm scripts, ES modules, Node child_process API	Do not imitate literally; choose the Python equivalent.
Not replaced by backend Python	DOM, window, browser fetch, chessboard.js callbacks	Keep JavaScript or replace with another browser technology.

6. Route registration and request parameters

6.1 Basic route

Express

```
// Express
app.get("/health", (req, res) => {
  res.json({ status: "ok" });
});
```

FastAPI

```
# FastAPI
@app.get("/health")
def health() -> dict[str, str]:
    return {"status": "ok"}
```

The idea transfers directly: method + path + handler. The mechanics differ. Express commonly sends through `res.json()`; FastAPI usually returns a Python value and lets response serialization occur after the function returns.

6.2 Path parameters

Express path parameter

```
// Express
app.get("/game/:id", (req, res) => {
  const gameId = Number(req.params.id);
  // manual validation is required
});
```

FastAPI path parameter

```
# FastAPI
from typing import Annotated
from fastapi import Path

@app.get("/game/{game_id}")
def get_game(game_id: Annotated[int, Path(gt=0)]):
    # game_id is already parsed as an int and constrained to > 0
    ...
```

FastAPI derives validation from Python annotations. A non-integer path value is rejected before the function body runs. This replaces a meaningful amount of repetitive request-boundary code.

6.3 Query parameters

Express query parameter

```
// Express
app.get("/games", (req, res) => {
```



```
const limit = Math.min(Number(req.query.limit ?? 50), 100);
});
```

FastAPI query parameter

```
# FastAPI
from typing import Annotated
from fastapi import Query

@app.get("/games")
def list_games(limit: Annotated[int, Query(ge=1, le=100)] = 50):
    ...
```

6.4 JSON request bodies and Pydantic

Express body parsing and manual validation

```
// Express
app.use(express.json());

app.post("/game/:id/move", (req, res) => {
  const uci = req.body?.uci;
  if (typeof uci !== "string") {
    return res.status(400).json({ error: "uci is required" });
  }
});
```

FastAPI request model

```
# FastAPI
from pydantic import BaseModel, ConfigDict, Field

class MoveRequest(BaseModel):
    model_config = ConfigDict(extra="forbid")
    uci: str = Field(pattern=r"^[a-h][1-8][a-h][1-8][qrbn]?$", )

@app.post("/game/{game_id}/move")
def submit_move(game_id: int, body: MoveRequest):
    uci = body.uci
    ...
```

FastAPI reads JSON, constructs the Pydantic model, validates types and constraints, and returns a structured validation error when the body is invalid. Pydantic validation is an API-boundary tool; it does not replace domain validation such as determining whether a syntactically valid UCI move is legal in the current position.

7. Response models and preserving the frontend contract

The frontend already expects specific fields. A rewrite is safest when the JSON contract remains identical, even if Python naming conventions differ. Pydantic aliases let internal Python use `snake_case` while serialized JSON preserves `gameId` or other existing `camelCase` names.

Preserving a camelCase response field

```
from pydantic import BaseModel, ConfigDict, Field

class NewGameResponse(BaseModel):
    model_config = ConfigDict(populate_by_name=True)
    game_id: int = Field(alias="gameId")

@app.post("/game/new", response_model=NewGameResponse)
def create_game(body: NewGameRequest) -> NewGameResponse:
    game_id = service.create_game(body.time_control)
    return NewGameResponse(game_id=game_id)

# JSON sent to the browser:
# {"gameId": 42}
```

Contract-first rule: Do not change `gameId` to `game_id`, moves from a space-delimited string to a list, or error shapes during the backend rewrite unless the frontend is updated intentionally and the change is tested as a versioned contract change.

8. Routers: Express Router and FastAPI APIRouter

Express Router

```
// Express
const router = express.Router();
router.post("/new", createGameHandler);
router.post("/:id/move", submitMoveHandler);
app.use("/game", router);
```

FastAPI APIRouter

```
# app/api/routes/games.py
from fastapi import APIRouter

router = APIRouter(prefix="/game", tags=["games"])

@router.post("/new")
def create_game(...):
    ...

@router.post("/{game_id}/move")
def submit_move(...):
    ...

# app/main.py
app.include_router(router)
```

The organizational idea is nearly identical: define a feature router and include it in the application. APIRouter additionally integrates dependencies, response models, status codes, tags, and OpenAPI metadata.

9. Middleware: same purpose, different protocol

9.1 Request timing

Express middleware

```
// Express
app.use(async (req, res, next) => {
  const started = performance.now();
  res.on("finish", () => {
    console.log(req.method, req.path, res.statusCode,
      performance.now() - started);
  });
  next();
});
```

FastAPI HTTP middleware

```
# FastAPI
from time import perf_counter
from fastapi import Request

@app.middleware("http")
async def add_timing(request: Request, call_next):
  started = perf_counter()
  response = await call_next(request)
  elapsed_ms = (perf_counter() - started) * 1000
  response.headers["X-Elapsed-Ms"] = f"{elapsed_ms:.1f}"
  return response
```

The purpose transfers: run code around every request. The Express `next()` callback is not copied. FastAPI middleware awaits `call_next(request)` and receives a response. Lower-level ASGI middleware is also available when tighter protocol control is required.

9.2 Middleware is not the right tool for every cross-cutting concern

- Use middleware for request IDs, timing, broad headers, CORS, and global request/response concerns.
- Use dependencies for authentication, authorization, database connections, and request-scoped services.
- Use exception handlers for consistent domain/API error translation.
- Use application services for business workflows; do not hide business sequencing in middleware.

10. Dependency injection: constructor injection versus FastAPI Depends

The browser architecture already uses constructor injection: PlayController receives state, view, API, board adapter, and clock. The same idea transfers to Python services. FastAPI Depends is an additional request-time mechanism for resolving dependencies such as repositories, connections, authenticated users, or service objects.

```
# Pure application class: framework-independent constructor injection
class GameService:
    def __init__(self, repository: GameRepository):
        self._repository = repository

# FastAPI integration: construct request-scoped service
from typing import Annotated
from fastapi import Depends

GameServiceDep = Annotated[GameService, Depends(get_game_service)]

@router.post("/{game_id}/move")
def submit_move(game_id: int, body: MoveRequest,
                service: GameServiceDep):
    return service.submit_move(game_id, body.uci)
```

Architectural distinction: FastAPI dependency injection should wire infrastructure to the HTTP boundary. It should not cause every domain class to import FastAPI. Keep GameService and repositories usable in ordinary unit tests without constructing an ASGI request.

11. Error handling

11.1 Express model

```
app.use((err, req, res, next) => {
  console.error(err);
  res.status(err.status ?? 500).json({
    error: err.message ?? "Internal server error"
  });
});
```

11.2 FastAPI model

```
from fastapi import Request
from fastapi.responses import JSONResponse

@app.exception_handler(GameNotFound)
async def game_not_found_handler(
    request: Request,
    exc: GameNotFound,
) -> JSONResponse:
    return JSONResponse(
        status_code=404,
        content={"error": str(exc)},
    )

@app.exception_handler(IllegalGameMove)
async def illegal_move_handler(
    request: Request,
    exc: IllegalGameMove,
) -> JSONResponse:
    return JSONResponse(
        status_code=422,
        content={"error": str(exc)},
    )
```

Both frameworks support centralized error translation. FastAPI can also raise HTTPException directly, but domain/application code is cleaner when it raises domain-specific exceptions and the API layer maps those exceptions to HTTP status codes.

12. Static files and HTML pages

Express

```
// Express
app.use(express.static(publicDirectory));
app.get("/puzzles", (req, res) => {
  res.sendFile(path.join(publicDirectory, "puzzles.html"));
});
```

FastAPI

```
# FastAPI
from pathlib import Path
from fastapi.responses import FileResponse
from fastapi.staticfiles import StaticFiles

PUBLIC_DIR = Path(__file__).resolve().parents[1] / "public"

app.mount(
    "/static",
    StaticFiles(directory=PUBLIC_DIR),
    name="static",
)

@app.get("/", include_in_schema=False)
def play_page() -> FileResponse:
    return FileResponse(PUBLIC_DIR / "index.html")

@app.get("/puzzles", include_in_schema=False)
def puzzle_page() -> FileResponse:
    return FileResponse(PUBLIC_DIR / "puzzles.html")
```

Mounting /static changes asset URLs unless the HTML is adjusted. An alternative is mounting the public directory at / after explicit API/page routes, but broad mounts and catch-all routes require careful ordering. During migration, choose the approach that minimizes frontend changes and verify every module, CSS, image, and board-piece URL in the browser Network panel.

13. Lifecycle: app.listen versus ASGI lifespan

Concern	Node/Express	FastAPI/Uvicorn
Socket listener	app.listen(...) creates/starts HTTP server	Uvicorn owns listener and invokes ASGI app
Route registration	app.get/app.post/app.use at startup	Decorators and include_router during import/app construction
Startup resources	Top-level code or explicit initialization before listen	Lifespan code before yield
Shutdown	Signal handlers/server close callbacks	Lifespan cleanup after yield
Multiple processes	Cluster/process manager/container replicas	Uvicorn workers/container replicas

14. Package/module systems

Node.js	Python	Important difference
package.json	pyproject.toml	Project metadata, dependencies, scripts/tool configuration use different ecosystems.
package-lock.json	uv.lock / poetry.lock / pinned requirements	Use one reproducible dependency strategy.
.js/.mjs module	.py module	Both execute top-level code when loaded/imported.
Directory package	Directory with Python package semantics	__init__.py commonly makes package intent explicit.

Node.js	Python	Important difference
import/export	import/from ... import	Python import paths depend on package execution context.
npm run dev	uvicorn/fastapi/uv/pip tooling	There is no need to recreate npm scripts exactly.

15. Things that do not transfer literally

Node/Express mechanism	Why literal translation is wrong	Python/FastAPI approach
req, res mutation	FastAPI is designed around typed parameters and returned values.	Declare parameters/models; return response model or Response.
next()	ASGI middleware/dependency execution has different control flow.	await call_next, Depends, or exception handlers.
Promise-first ecosystem	Python coroutines and synchronous functions coexist differently.	Choose def or async def based on I/O behavior.
Single JS event loop assumptions	Uvicorn can use multiple workers; sync routes use threadpool.	Model process, thread, event-loop, and resource ownership explicitly.
child_process API	Python subprocess APIs and python-chess abstractions differ.	Use EngineManager, asyncio.subprocess, or python-chess engine module.
Shared TypeScript types	Python models do not automatically type browser JS.	Generate OpenAPI clients/types or maintain explicit DTO contract tests.
Browser JavaScript replacement	Server Python cannot manipulate a remote DOM.	Keep browser JS or adopt a browser framework.

Part III - Recommended Python architecture and file structures

16. Start with a minimal migration structure

The first objective should be behavioral compatibility, not maximum abstraction. A small structure lets you replace endpoints quickly while preventing a new monolithic main.py from becoming the Python version of an oversized server.js.

Minimal migration-oriented structure

```

ichigo-chess/
├── app/
│   ├── __init__.py
│   ├── main.py
│   ├── database.py
│   ├── schemas.py
│   ├── game_service.py
│   ├── game_repository.py
│   ├── engine.py
│   └── routes/
│       ├── __init__.py
│       ├── games.py
│       ├── engine.py
│       └── puzzles.py
├── public/
│   ├── index.html
│   ├── puzzles.html
│   ├── css/
│   └── js/
├── data/
│   └── ichigo.db
├── tests/
├── pyproject.toml
└── README.md

```

17. Evolve toward a layered feature structure

After endpoint compatibility is established, split responsibilities by reason to change. The following structure is suitable for the Ichigo feature set without requiring a heavyweight framework or service mesh.

Recommended layered structure

```

ichigo-chess/
├── app/
│   ├── __init__.py
│   ├── main.py           # application composition root
│   ├── config.py         # settings loaded from environment
│   └── api/
│       ├── dependencies.py # request-scoped wiring
│       ├── exception_handlers.py # domain error -> HTTP mapping
│       └── routes/
│           ├── games.py
│           ├── engine.py
│           ├── puzzles.py
│           └── chesscom.py
├── domain/
│   ├── errors.py
│   ├── game.py
│   └── puzzle.py
├── schemas/
│   ├── game.py           # Pydantic HTTP DTOs
│   ├── engine.py
│   └── puzzle.py
├── services/
│   ├── game_service.py
│   ├── engine_service.py
│   └── puzzle_service.py
├── repositories/
└── protocols.py

```

```

├── sqlite_game_repository.py
├── infrastructure/
│   ├── database.py
│   ├── stockfish.py
│   ├── chesscom_client.py
│   └── model_loader.py
├── public/
├── data/
├── models/
├── tests/
│   ├── unit/
│   └── integration/
│       └── contract/
├── pyproject.toml
└── README.md

```

18. Responsibility map

Area	Owns	Must not own
api/routes	HTTP methods, paths, dependency declaration, response models/status	Chess reconstruction algorithm or SQL details
schemas	External request/response shapes and validation	Database records or core business workflows
services	Use-case sequencing and transaction intent	FastAPI Request objects or DOM behavior
domain	Business concepts, outcomes, domain exceptions	HTTP status codes or SQLite cursor objects
repositories	Persistence interface and storage operations	HTTP serialization or board rendering
infrastructure	SQLite, Stockfish, HTTP clients, model files	User-interface decisions
main.py	Composition, routers, lifespan, middleware, exception handlers	Feature algorithms

19. Dependency direction

```

HTTP request
  v
API route + Pydantic DTO
  v
Application service
  v
Domain logic / python-chess
  v
Repository protocol or infrastructure adapter
  v
SQLite / Stockfish / Chess.com / model

```

Dependencies point inward toward stable application concepts.
Infrastructure is supplied to services, not imported everywhere.

19.1 Repository protocol

```

from typing import Protocol

from app.domain.game import GameRecord

class GameRepository(Protocol):
    def create(self, time_control: str) -> int:
        ...

    def get(self, game_id: int) -> GameRecord | None:
        ...

    def save_state(
        self,
        game_id: int,

```

```

*,
moves: str,
result: str,
pgn: str | None,
) -> None:
...

```

The service can be tested against an in-memory fake implementing the protocol. The SQLite implementation can change without rewriting the service. This is the Python equivalent of injecting a repository or database adapter into a Node service/router factory.

20. Configuration

```

# app/config.py
from functools import lru_cache
from pathlib import Path

from pydantic_settings import BaseSettings, SettingsConfigDict

class Settings(BaseSettings):
    model_config = SettingsConfigDict(
        env_prefix="ICHIGO_",
        env_file=".env",
        extra="ignore",
    )

    database_path: Path = Path("data/ichigo.db")
    public_dir: Path = Path("public")
    stockfish_path: Path = Path("/usr/local/bin/stockfish")
    engine_timeout_seconds: float = 10.0
    chesscom_timeout_seconds: float = 12.0
    model_path: Path = Path("models/puzzle_ranker.joblib")

    @lru_cache
    def get_settings() -> Settings:
        return Settings()

```

Configuration should come from environment or explicit config, not scattered constants. Paths and timeouts are operational choices. Secrets such as API credentials must never be embedded in frontend JavaScript or committed to source control.

21. Database lifecycle and request-scoped connections

```

# app/infrastructure/database.py
import sqlite3
from collections.abc import Iterator
from pathlib import Path

SCHEMA = """
CREATE TABLE IF NOT EXISTS games (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    moves TEXT NOT NULL DEFAULT "",
    result TEXT NOT NULL DEFAULT '*',
    pgn TEXT,
    time_control TEXT NOT NULL,
    created_at TEXT NOT NULL DEFAULT CURRENT_TIMESTAMP
);
"""

def initialize_database(path: Path) -> None:
    path.parent.mkdir(parents=True, exist_ok=True)
    with sqlite3.connect(path) as connection:
        connection.executescript(SCHEMA)

def open_connection(path: Path) -> sqlite3.Connection:
    connection = sqlite3.connect(path, check_same_thread=False)
    connection.row_factory = sqlite3.Row
    connection.execute("PRAGMA foreign_keys = ON")
    return connection

# app/api/dependencies.py
from collections.abc import Iterator
import sqlite3

```



```

from typing import Annotated

from fastapi import Depends

from app.config import Settings, get_settings
from app.infrastructure.database import open_connection

SettingsDep = Annotated[Settings, Depends(get_settings)]

def get_connection(settings: SettingsDep) -> Iterator[sqlite3.Connection]:
    connection = open_connection(settings.database_path)
    try:
        yield connection
    finally:
        connection.close()

ConnectionDep = Annotated[sqlite3.Connection, Depends(get_connection)]

```

A connection-per-request dependency gives the repository an explicit transaction boundary and avoids one global SQLite connection being shared unpredictably across concurrent request threads. `check_same_thread=False` permits the connection to cross FastAPI threadpool boundaries; the connection is still request-scoped and must not be stored for reuse by other requests.

22. When to introduce SQLAlchemy

Stay with sqlite3 when...	Introduce SQLAlchemy when...
The schema is small and SQL is straightforward.	The domain has many related tables and migrations.
You want maximum visibility into SQL and transactions.	You need database portability or a unit-of-work/session abstraction.
The current better-sqlite3 design maps cleanly to prepared SQL.	Query composition and relationship loading are becoming repetitive.
You are still proving endpoint compatibility.	The team understands SQLAlchemy 2.x session and async/sync choices.

Do not adopt an ORM merely because the language changed: Changing Node to Python and raw SQLite to a new ORM simultaneously increases the number of moving parts. First preserve behavior; then make a separate evidence-based persistence decision.

Part IV - End-to-end implementation slices with code

23. Application composition root

```
# app/main.py
from contextlib import asynccontextmanager

import httpx
from fastapi import FastAPI

from app.api.exception_handlers import install_exception_handlers
from app.api.routes import chesscom, engine, games, puzzles, pages
from app.config import get_settings
from app.infrastructure.database import initialize_database
from app.infrastructure.stockfish import EngineManager

@asynccontextmanager
async def lifespan(app: FastAPI):
    settings = get_settings()
    initialize_database(settings.database_path)

    app.state.http_client = httpx.AsyncClient(
        timeout=settings.chesscom_timeout_seconds,
        headers={"User-Agent": "IchigoChess/1.0"},
    )

    app.state.engine = EngineManager(settings.stockfish_path)
    await app.state.engine.start()

    try:
        yield
    finally:
        await app.state.engine.close()
        await app.state.http_client.aclose()

def create_app() -> FastAPI:
    app = FastAPI(
        title="Ichigo Chess API",
        version="1.0.0",
        lifespan=lifespan,
    )

    app.include_router(games.router)
    app.include_router(engine.router)
    app.include_router(puzzles.router)
    app.include_router(chesscom.router)
    app.include_router(pages.router)

    install_exception_handlers(app)
    return app

app = create_app()
```

This file is a composition root. It knows concrete implementations and wires them together. Route modules should not independently create HTTP clients, databases, engines, or models. Central ownership makes startup, shutdown, testing, and resource-count reasoning visible.

24. Vertical slice A: create a game

24.1 Request and response schemas

```
# app/schemas/game.py
from pydantic import BaseModel, ConfigDict, Field

class NewGameRequest(BaseModel):
    model_config = ConfigDict(extra="forbid")
    time_control: str = Field(min_length=1, max_length=32)

class NewGameResponse(BaseModel):
```

```
model_config = ConfigDict(populate_by_name=True)
game_id: int = Field(alias="gameId")
```

24.2 Repository implementation

```
# app/repositories/sqlite_game_repository.py
import sqlite3

from app.domain.game import GameRecord

class SQLiteGameRepository:
    def __init__(self, connection: sqlite3.Connection):
        self._connection = connection

    def create(self, time_control: str) -> int:
        cursor = self._connection.execute(
            "INSERT INTO games (time_control) VALUES (?)",
            (time_control,),
        )
        self._connection.commit()
        return int(cursor.lastrowid)

    def get(self, game_id: int) -> GameRecord | None:
        row = self._connection.execute(
            """
            SELECT id, moves, result, pgn, time_control
            FROM games
            WHERE id = ?
            """
            ,
            (game_id,),
        ).fetchone()
        if row is None:
            return None
        return GameRecord(
            id=int(row["id"]),
            moves=str(row["moves"]),
            result=str(row["result"]),
            pgn=row["pgn"],
            time_control=str(row["time_control"]),
        )
```

24.3 Service

```
# app/services/game_service.py
class GameService:
    def __init__(self, repository: GameRepository):
        self._repository = repository

    def create_game(self, time_control: str) -> int:
        normalized = time_control.strip()
        if not normalized:
            raise InvalidTimeControl("time control cannot be empty")
        return self._repository.create(normalized)
```

24.4 Dependency wiring and route

```
# app/api/dependencies.py
from typing import Annotated
from fastapi import Depends

from app.repositories.sqlite_game_repository import SQLiteGameRepository
from app.services.game_service import GameService

def get_game_service(connection: ConnectionDep) -> GameService:
    repository = SQLiteGameRepository(connection)
    return GameService(repository)

GameServiceDep = Annotated[GameService, Depends(get_game_service)]

# app/api/routes/games.py
from fastapi import APIRouter, status

from app.api.dependencies import GameServiceDep
from app.schemas.game import NewGameRequest, NewGameResponse
```

```

router = APIRouter(prefix="/game", tags=["games"])

@router.post(
    "/new",
    response_model=NewGameResponse,
    status_code=status.HTTP_201_CREATED,
)
def create_game(
    body: NewGameRequest,
    service: GameServiceDep,
) -> NewGameResponse:
    game_id = service.create_game(body.time_control)
    return NewGameResponse(game_id=game_id)

```

If the existing frontend assumes HTTP 200 rather than 201, preserve 200 during migration. A status-code improvement is still a contract change and should be made deliberately.

25. Vertical slice B: submit and persist a legal move

25.1 Domain records and errors

```

# app/domain/game.py
from dataclasses import dataclass

@dataclass(frozen=True, slots=True)
class GameRecord:
    id: int
    moves: str
    result: str
    pgn: str | None
    time_control: str

@dataclass(frozen=True, slots=True)
class MoveOutcome:
    moves: str
    fen: str
    over: bool
    reason: str | None
    result: str
    pgn: str | None

# app/domain/errors.py
class IchigoError(Exception):
    pass

class GameNotFound(IchigoError):
    pass

class CorruptGameHistory(IchigoError):
    pass

class IllegalGameMove(IchigoError):
    pass

```

25.2 Chess reconstruction helpers

```

# app/services/chess_rules.py
import chess
import chess.pgn

from app.domain.errors import CorruptGameHistory, IllegalGameMove

def board_from_uci_history(history: str) -> chess.Board:
    board = chess.Board()
    for token in history.split():
        try:
            move = chess.Move.from_uci(token)
        except ValueError as exc:
            raise CorruptGameHistory(
                f"invalid stored UCI token: {token}"
            ) from exc

    if move not in board.legal_moves:

```

```

        raise CorruptGameHistory(
            f"stored move is illegal at replay point: {token}"
        )
    board.push(move)
    return board

def parse_legal_move(board: chess.Board, uci: str) -> chess.Move:
    try:
        move = chess.Move.from_uci(uci)
    except ValueError as exc:
        raise IllegalGameMove(f"invalid UCI move: {uci}") from exc

    if move not in board.legal_moves:
        raise IllegalGameMove(f"illegal move: {uci}")
    return move

def export_pgn(board: chess.Board) -> str:
    game = chess.pgn.Game.from_board(board)
    exporter = chess.pgn.StringExporter(
        headers=True,
        variations=False,
        comments=False,
    )
    return game.accept(exporter)

```

25.3 Atomic repository update

```

# method on SQLiteGameRepository

def save_state(
    self,
    game_id: int,
    *,
    moves: str,
    result: str,
    pgn: str | None,
) -> None:
    try:
        self._connection.execute("BEGIN IMMEDIATE")
        cursor = self._connection.execute(
            """
            UPDATE games
            SET moves = ?, result = ?, pgn = ?
            WHERE id = ?
            """
            ,
            (moves, result, pgn, game_id),
        )
        if cursor.rowcount != 1:
            raise GameNotFound(f"game {game_id} not found")
        self._connection.commit()
    except Exception:
        self._connection.rollback()
        raise

```

25.4 Service orchestration

```

# method on GameService
import chess

from app.services.chess_rules import (
    board_from_uci_history,
    export_pgn,
    parse_legal_move,
)

def submit_move(self, game_id: int, uci: str) -> MoveOutcome:
    record = self._repository.get(game_id)
    if record is None:
        raise GameNotFound(f"game {game_id} not found")

    board = board_from_uci_history(record.moves)
    move = parse_legal_move(board, uci)

```

```

board.push(move)

moves = " ".join(m.uci() for m in board.move_stack)
outcome = board.outcome(claim_draw=True)

if outcome is None:
    result = "*"
    reason = None
    pgn = None
else:
    result = outcome.result()
    reason = outcome.termination.name.lower()
    pgn = export_pgn(board)

self._repository.save_state(
    game_id,
    moves=moves,
    result=result,
    pgn=pgn,
)

return MoveOutcome(
    moves=moves,
    fen=board.fen(),
    over=outcome is not None,
    reason=reason,
    result=result,
    pgn=pgn,
)

```

25.5 HTTP response

```

# app/schemas/game.py
from typing import Literal
from pydantic import BaseModel, ConfigDict, Field

class MoveRequest(BaseModel):
    model_config = ConfigDict(extra="forbid")
    uci: str = Field(pattern=r"^[a-h][1-8][a-h][1-8][qrbn]?$", )

class MoveResponse(BaseModel):
    ok: bool = True
    moves: str
    fen: str
    over: bool
    reason: str | None
    result: Literal["*", "1-0", "0-1", "1/2-1/2"]
    pgn: str | None

@router.post("/{game_id}/move", response_model=MoveResponse)
def submit_move(
    game_id: Annotated[int, Path(gt=0)],
    body: MoveRequest,
    service: GameServiceDep,
) -> MoveResponse:
    outcome = service.submit_move(game_id, body.uci)
    return MoveResponse(**outcome.__dict__)

```

For a slots dataclass, `outcome.__dict__` is unavailable. In production code, construct the response explicitly or use `dataclasses.asdict(outcome)`. The explicit form is clearest and avoids accidental schema coupling:

Correct explicit conversion

```

return MoveResponse(
    moves=outcome.moves,
    fen=outcome.fen,
    over=outcome.over,
    reason=outcome.reason,
    result=outcome.result,
    pgn=outcome.pgn,
)

```

Critical correctness boundary: The browser may optimistically project a move, but python-chess remains the authoritative legality engine. The server response must remain the reconciliation source for FEN, history, result, and

PGN.

26. Exception handlers that preserve a simple error contract

```
# app/api/exception_handlers.py
from fastapi import FastAPI, Request
from fastapi.responses import JSONResponse

from app.domain.errors import (
    CorruptGameHistory,
    GameNotFound,
    IllegalGameMove,
)

def install_exception_handlers(app: FastAPI) -> None:
    @app.exception_handler(GameNotFound)
    async def handle_not_found(
        request: Request,
        exc: GameNotFound,
    ) -> JSONResponse:
        return JSONResponse(status_code=404, content={"error": str(exc)})

    @app.exception_handler(IllegalGameMove)
    async def handle_illegal_move(
        request: Request,
        exc: IllegalGameMove,
    ) -> JSONResponse:
        return JSONResponse(status_code=422, content={"error": str(exc)})

    @app.exception_handler(CorruptGameHistory)
    async def handle_corruption(
        request: Request,
        exc: CorruptGameHistory,
    ) -> JSONResponse:
        return JSONResponse(status_code=409, content={"error": str(exc)})
```

27. Vertical slice C: engine request

Stockfish is still a separate process. Python makes the adapter easier because python-chess already understands UCI, but the process boundary does not disappear. A single engine process is a constrained resource and should have explicit ownership, timeout, serialization, and shutdown behavior.

```
# app/infrastructure/stockfish.py
import asyncio
from pathlib import Path

import chess
import chess.engine

class EngineManager:
    def __init__(self, executable: Path):
        self._executable = executable
        self._engine: chess.engine.SimpleEngine | None = None
        self._lock = asyncio.Lock()

    async def start(self) -> None:
        if self._engine is not None:
            return
        self._engine = await asyncio.to_thread(
            chess.engine.SimpleEngine.popen_uci,
            str(self._executable),
        )

    async def best_move(
        self,
        board: chess.Board,
        *,
        movetime_ms: int,
    ) -> str:
```

```

if self._engine is None:
    raise RuntimeError("engine has not been started")

limit = chess.engine.Limit(time=movetime_ms / 1000)
async with self._lock:
    result = await asyncio.to_thread(
        self._engine.play,
        board,
        limit,
    )

if result.move is None:
    raise RuntimeError("engine returned no move")
return result.move.uci()

async def close(self) -> None:
    if self._engine is None:
        return
    engine = self._engine
    self._engine = None
    await asyncio.to_thread(engine.quit)

```

27.1 Why `asyncio.to_thread` appears here

SimpleEngine methods are synchronous from the caller's perspective. Calling a blocking method directly inside an async path operation would block the Uvicorn event loop. `asyncio.to_thread` moves the blocking wait to a worker thread. The lock prevents overlapping commands from treating one engine process as if it were an unlimited concurrent service.

27.2 Engine route dependency

```

# app/api/dependencies.py
from fastapi import Request

def get_engine(request: Request) -> EngineManager:
    return request.app.state.engine

EngineDep = Annotated[EngineManager, Depends(get_engine)]

# app/api/routes/engine.py
from fastapi import APIRouter
from pydantic import BaseModel, Field

router = APIRouter(tags=["engine"])

class BestMoveRequest(BaseModel):
    moves: str = ""
    movetimeMs: int = Field(default=250, ge=10, le=30_000)

class BestMoveResponse(BaseModel):
    bestmove: str

@router.post("/bestmove", response_model=BestMoveResponse)
async def best_move(
    body: BestMoveRequest,
    engine: EngineDep,
) -> BestMoveResponse:
    board = board_from_uci_history(body.moves)
    uci = await engine.best_move(
        board,
        movetime_ms=body.movetimeMs,
    )
    return BestMoveResponse(bestmove=uci)

```

The request field `movetimeMs` is kept because the frontend already sends it. A later API version could use `snake_case` internally with aliases, but compatibility comes first.

28. Vertical slice D: Chess.com integration

External HTTP calls are true asynchronous I/O and are a good use of `async def` plus an async HTTP client. Create one client per worker during lifespan so connections can be pooled and closed cleanly.


```
# app/infrastructure/chesscom_client.py
import httpx

class ChessComClient:
    BASE_URL = "https://api.chess.com/pub"

    def __init__(self, client: httpx.AsyncClient):
        self._client = client

    async def get_archives(self, username: str) -> list[str]:
        url = f"{self.BASE_URL}/player/{username}/games/archives"
        response = await self._client.get(url)
        response.raise_for_status()
        data = response.json()
        return list(data.get("archives", []))

    async def get_archive_games(self, archive_url: str) -> list[dict]:
        response = await self._client.get(archive_url)
        response.raise_for_status()
        data = response.json()
        return list(data.get("games", []))

# dependency
from fastapi import Request

def get_chesscom_client(request: Request) -> ChessComClient:
    return ChessComClient(request.app.state.http_client)
```

28.1 Timeout and trust-boundary rules

- Normalize and validate the username before placing it into a URL path.
- Set total/connect/read timeouts; never allow an external service to hang a request indefinitely.
- Limit archive and game counts to prevent unexpectedly large analysis workloads.
- Treat external JSON as untrusted and normalize it into your own DTOs.
- Map upstream 404, rate-limit, timeout, and malformed-response failures to meaningful local errors.

29. Vertical slice E: puzzle/ML scoring

29.1 What becomes simpler

```
# Node-oriented boundary
Node route
-> spawn Python scoring script
-> serialize features to stdin/file
-> parse stdout
-> handle process exit and protocol errors

# Python-oriented boundary
FastAPI service
-> model_adapter.score(features)
-> receive Python values directly
```

Removing the Node-to-Python scoring subprocess can eliminate serialization and protocol code. However, loading the model inside the web process couples model memory and failures to every API worker. Simpler source code is not automatically simpler operations.

29.2 Lifespan-managed model adapter

```
# app/infrastructure/model_loader.py
import asyncio
from pathlib import Path

class PuzzleRanker:
    def __init__(self, model):
        self._model = model

    @classmethod
    async def load(cls, path: Path) -> "PuzzleRanker":
        model = await asyncio.to_thread(load_model_from_disk, path)
        return cls(model)
```

```

async def score(self, features: list[float]) -> float:
    value = await asyncio.to_thread(
        self._model.predict_score,
        features,
    )
    return float(value)

```

Whether a thread is appropriate depends on the model library. NumPy/PyTorch operations often execute native code and may release the GIL, but inference still consumes CPU/GPU and blocks the calling coroutine until complete. Benchmark realistic concurrency. For long or high-volume jobs, move the work to dedicated workers rather than adding more web workers blindly.

30. Serving the retained frontend

30.1 HTML paths

```

# app/api/routes/pages.py
from pathlib import Path
from fastapi import APIRouter
from fastapi.responses import FileResponse

router = APIRouter(include_in_schema=False)
PUBLIC_DIR = Path("public").resolve()

@router.get("/")
def play_page() -> FileResponse:
    return FileResponse(PUBLIC_DIR / "index.html")

@router.get("/puzzles")
def puzzle_page() -> FileResponse:
    return FileResponse(PUBLIC_DIR / "puzzles.html")

```

30.2 Static mount

```

from fastapi.staticfiles import StaticFiles

app.mount(
    "/static",
    StaticFiles(directory=settings.public_dir),
    name="static",
)

```

30.3 HTML asset paths

```

<link rel="stylesheet" href="/static/css/base.css">
<link rel="stylesheet" href="/static/css/play.css">
<script type="module" src="/static/js/play/play.mjs"></script>

```

If you preserve the current URLs exactly, mount or route files accordingly. Do not assume a successful GET / proves module loading works; verify every imported .mjs file and third-party board asset.

31. CORS only when origins differ

```

from fastapi.middleware.cors import CORSMiddleware

app.add_middleware(
    CORSMiddleware,
    allow_origins=["http://localhost:5173"],
    allow_credentials=True,
    allow_methods=["GET", "POST"],
    allow_headers=["Content-Type"],
)

```

Do not use `allow_origins=["*"]` together with credentialed cookie authentication. Same-origin serving avoids this configuration entirely and reduces development mismatch.

Part V - Concurrency, engines, ML, and long-running work

32. Node event loop versus FastAPI/Uvicorn execution

Aspect	Node.js process	Typical FastAPI/Uvicorn worker
Primary async model	JavaScript event loop and promises	asyncio event loop and coroutines
Synchronous handler code	Runs on JS main thread and blocks it	def path operations are run in a threadpool
async handler blocking call	Blocks if synchronous CPU/I/O is used	Also blocks event loop if synchronous call is made directly
CPU parallelism	Use workers/child processes/native code	Use workers/process pools/native code; GIL matters for Python bytecode
Multiple server workers	Separate processes via cluster/process manager	Separate Uvicorn processes with independent memory
Shared in-memory state	Only within one process	Only within one worker process

33. Choosing def or async def in FastAPI

Use	When	Ichigo examples
def	The implementation calls blocking/synchronous libraries and work is reasonably short.	Raw sqlite3 route, simple python-chess validation, file operations.
async def	The implementation awaits async libraries or coordinates async resources.	htpx Chess.com calls, EngineManager async wrapper, WebSocket endpoint.
Dedicated worker/job	The task is long, CPU-heavy, retryable, or should survive web-process restart.	Analyze many games, deep engine search, batch puzzle generation.

Common mistake: Writing async def does not make synchronous code non-blocking. An async route that calls `time.sleep()`, `sqlite3` work with large queries, `model.predict()`, or `engine.play()` directly can freeze that worker's event loop until the call returns.

34. Four kinds of work

Work type	Example	Preferred mechanism
Fast synchronous	Validate one UCI move and write one SQLite row	def route or sync service in threadpool
Async network I/O	Fetch Chess.com archive JSON	async def + <code>htpx.AsyncClient</code>
Blocking external process call	Wait for Stockfish best move	Async adapter + <code>to_thread/async subprocess</code> + lock
Long CPU/engine pipeline	Analyze 50 games and score candidates	Durable job worker and persisted job state

35. FastAPI BackgroundTasks: useful but limited

```
from fastapi import BackgroundTasks

@router.post("/game/{game_id}/finish")
def finish_game(
    game_id: int,
    background_tasks: BackgroundTasks,
```

```

):
    result = service.finish_game(game_id)
    background_tasks.add_task(write_audit_log, game_id, result.result)
    return result

```

This is suitable for small best-effort work after the response. It is not a durable queue: the task runs in the web process, may be lost if the process stops, competes with request capacity, and does not provide distributed retries, scheduling, progress, or independent scaling.

36. Recommended long-analysis job architecture

```

Browser
-> POST /analysis-jobs
   body: username, maxGames, settings
<- 202 Accepted {jobId, statusUrl}

Worker process
-> reads queued job
-> Chess.com retrieval
-> engine analysis
-> feature/model scoring
-> stores progress and result

Browser
-> GET /analysis-jobs/{jobId}
<- {status, progress, puzzles?, error?}

```

This decouples HTTP request duration from analysis duration and supports cancellation, retries, progress, and separate resource sizing. A simple first implementation can use a database-backed jobs table and one worker process; a production system can later use a dedicated queue technology.

37. Multi-worker resource multiplication

Resource	With 1 worker	With 4 workers
FastAPI application	1 process	4 independent processes
In-memory cache	1 separate cache	4 divergent caches unless externalized
Stockfish manager	Typically 1 engine process	Typically 4 engine processes unless separated
ML model	1 loaded copy	4 loaded copies (subject to OS/runtime sharing details)
SQLite access	One process issuing connections	Four processes contending on one file

Deployment implication: Do not set `--workers 4` merely because the laptop has multiple cores. Count engine memory, model memory, SQLite write contention, and expected request mix. For engine-heavy systems, a small web tier plus a separate engine worker pool is often clearer than one engine per web worker.

38. Cancellation and stale work

The browser already uses `AbortController` and generation tokens to prevent stale engine or puzzle responses from overwriting new state. That design remains valid. Server-side cancellation is harder: a client disconnect does not automatically guarantee that a blocking engine call or external process stops. Time limits, job IDs, cancellation flags, and subprocess termination policies must be explicit for long work.

39. State ownership after migration

State	Authoritative owner	Why
-------	---------------------	-----

State	Authoritative owner	Why
Legal game position/history	FastAPI service + persisted SQLite state	Prevents browser projection from becoming a second rule engine.
Board pixels and selection	Browser	Server cannot directly render remote DOM.
Engine process state	EngineManager or engine worker	Must be serialized and lifecycle-managed.
Long job status	Persistent job store	Must survive browser refresh and worker restart.
Request-scoped DB transaction	Repository/dependency	Must end in commit or rollback per use case.

Part VI - Testing, observability, security, and deployment

40. Test pyramid for the Python backend

Layer	Test target	Examples
Domain/unit	Pure helpers and services with fakes	History replay, legal/illegal moves, result mapping, time-control validation.
Repository	SQLite implementation against temp DB	Create/get/update, transaction rollback, missing ID.
API integration	FastAPI app + dependency overrides	Status, JSON shape, validation, error mapping.
Contract	Exact compatibility with frontend expectations	gameld alias, moves string, 422 error body, FEN field.
Browser E2E	Real frontend + Python server	New game, click move, drag move, rollback, daily puzzle.

41. Service unit test with a fake repository

```
# tests/unit/test_game_service.py
from app.domain.game import GameRecord
from app.services.game_service import GameService

class FakeGameRepository:
    def __init__(self):
        self.games = {
            1: GameRecord(1, "", "*", None, "300+0")
        }
        self.saved = None

    def get(self, game_id: int):
        return self.games.get(game_id)

    def save_state(self, game_id: int, **state):
        self.saved = (game_id, state)

def test_submit_e2e4_returns_authoritative_fen():
    repository = FakeGameRepository()
    service = GameService(repository)

    outcome = service.submit_move(1, "e2e4")

    assert outcome.moves == "e2e4"
    assert outcome.fen.split()[1] == "b"
    assert outcome.over is False
    assert repository.saved[0] == 1
```

42. API test with dependency override

```
from fastapi.testclient import TestClient

from app.api.dependencies import get_game_service
from app.main import create_app

class StubGameService:
    def create_game(self, time_control: str) -> int:
        assert time_control == "300+0"
        return 42

def test_create_game_contract():
    app = create_app()
    app.dependency_overrides[get_game_service] = (
        lambda: StubGameService()
    )

    with TestClient(app) as client:
        response = client.post(
            "/game/new",
            json={"time_control": "300+0"},
        )

    assert response.status_code in {200, 201}
    assert response.json() == {"gameId": 42}
```

For tests that do not need Stockfish or external clients, provide a testing lifespan or override resource dependencies so the test suite does not launch heavy processes. Application factories make this easier.

43. Contract regression tests before migration

Capture representative responses from the Node backend before rewriting. These are not necessarily golden forever, but they expose accidental contract drift.

- POST /game/new success and invalid body.
- POST /game/:id/move legal, illegal, missing game, malformed UCI, and terminal game.
- POST /game/:id/finish and PGN retrieval.
- POST /bookmove with hit and miss.
- POST /bestmove including timeout/failure behavior.
- GET /puzzles/daily and /puzzles/random DTO shapes.
- Chess.com user not found, timeout, malformed upstream response, and normal results.

44. Observability

Signal	Include	Purpose
Request log	request ID, method, path, status, elapsed time	Trace browser/API behavior.
Engine log	request ID, FEN hash, limit, elapsed time, termination	Separate engine latency from web latency.
External HTTP log	upstream host/path category, status, elapsed, retry	Diagnose Chess.com failures.
Database log	operation name, transaction outcome, elapsed	Detect contention and slow queries without leaking data.
Job log	job ID, stage, progress, failure class	Recover and explain long analyses.

44.1 Request ID middleware

```
from uuid import uuid4
from fastapi import Request

@app.middleware("http")
async def request_id_middleware(request: Request, call_next):
```

```
request_id = request.headers.get("X-Request-ID", str(uuid4()))
request.state.request_id = request_id
response = await call_next(request)
response.headers["X-Request-ID"] = request_id
return response
```

45. Security and trust boundaries

- Validate all path, query, body, PGN, FEN, UCI, username, and limit inputs.
- Use parameterized SQL; never interpolate user values into SQL strings.
- Limit request body size, game count, PGN length, engine depth/time, and puzzle count.
- Set external HTTP and engine timeouts and map failures without exposing internal stack traces.
- Keep secrets and filesystem paths out of response bodies and frontend code.
- When cookie authentication is introduced, design CSRF protection; CORS is not CSRF protection.
- Authorize game ownership on the server; hiding a button in the browser is not authorization.
- Serve a deliberate Content Security Policy after local/CDN dependencies are understood.

46. Development and production commands

```
# Create virtual environment
python -m venv .venv
source .venv/bin/activate

# Install project (example)
pip install -e ".[dev]"

# Development with reload
uvicorn app.main:app --host 127.0.0.1 --port 8080 --reload

# Production-like single worker
uvicorn app.main:app --host 0.0.0.0 --port 8080

# Multiple workers only after resource review
uvicorn app.main:app --host 0.0.0.0 --port 8080 --workers 2
```

47. Example pyproject.toml

```
[project]
name = "ichigo-chess"
version = "0.1.0"
requires-python = ">=3.12"
dependencies = [
    "fastapi",
    "uvicorn[standard]",
    "pydantic-settings",
    "python-chess",
    "httpx",
]

[project.optional-dependencies]
dev = [
    "pytest",
    "pytest-asyncio",
    "ruff",
    "mypy",
]

[tool.pytest.ini_options]
testpaths = ["tests"]

[tool.ruff]
line-length = 88
```

48. Deployment shapes

Shape	Use	Tradeoff
-------	-----	----------

Shape	Use	Tradeoff
Single process	Local development, personal deployment, early testing	Simplest; one failure domain and limited parallelism.
Multiple Uvicorn workers	I/O-heavy API with small per-worker resources	Memory/resources duplicate; SQLite/engine contention.
Web tier + engine worker	Engine-heavy play or analysis	More moving parts, but resource ownership is explicit.
Web tier + job queue + workers	Long personalized puzzle generation	Best reliability/scaling; requires persisted job model.
Separate frontend hosting	CDN/static hosting and API deployment	Requires CORS, deployment coordination, and absolute API configuration.

Part VII - Incremental migration and framework alternatives

49. Recommended migration sequence

Phase	Work	Exit criterion
0. Baseline	Freeze endpoint inventory, DTOs, statuses, DB schema, and browser flows.	Contract fixtures and E2E smoke tests exist.
1. Skeleton	Create FastAPI app, health route, static/page serving, config, test harness.	Frontend loads from Python server.
2. Game creation/read	Implement /game/new, /game/:id, /games.	Frontend can start and inspect games.
3. Move/finish	Implement legal move, rollback statuses, finish, PGN.	Complete human-vs-human persistence flow passes.
4. Engine	Add book and best-move adapters with lifecycle/timeouts.	Human-vs-engine flow passes.
5. Public puzzles	Daily/random and canonical puzzle DTO.	Daily/random puzzle E2E passes.
6. Chess.com	Async client, normalization, recent games.	External failures and success cases tested.
7. ML pipeline	Direct model adapter or job architecture.	Personalized puzzles pass load/performance tests.
8. Cutover	Run browser E2E, compare logs/data, switch port/service.	Node server is no longer required for normal operation.

50. Preserve the database or migrate it?

Both better-sqlite3 and Python sqlite3 use the same SQLite file format. You can often retain the existing database file. Before doing so, inspect schema, constraints, migration history, journal mode, and stored move format. Back up the file and run read-only compatibility tests before allowing the Python service to write production data.

50.1 Safe database cutover

1. Stop the Node writer or put the application into maintenance/read-only mode.
2. Copy the SQLite file and verify integrity.
3. Run the Python repository integration suite against the copy.
4. Compare representative game rows, PGNs, and move reconstruction.
5. Start only one writer service during cutover; do not let Node and Python race to update the same game.

6. Retain a rollback copy until the Python service is stable.

51. Running Node and Python side-by-side during migration

Two servers cannot both bind the same host port. During development, run Node on 8080 and FastAPI on 8081, or place a reverse proxy in front and route selected paths to Python. The simplest approach is usually to test the Python API directly on a separate port, then switch the frontend origin only during migration testing.

```
Node API:    http://localhost:8080
FastAPI API: http://localhost:8081
```

```
Contract test sends the same request to both:
POST /game/new
{"time_control": "300+0"}
```

Compare status and JSON shape, not generated database IDs.

52. Why FastAPI is the best default for this project

- The system is API-centric and already uses browser fetch with JSON.
- Pydantic makes request and response contracts explicit.
- Automatic OpenAPI documentation helps inspect and test endpoints.
- Dependency injection supports request-scoped connections and adapters.
- Async support fits external HTTP and coordinated engine adapters.
- Python integrates naturally with python-chess and the existing ML pipeline.
- APIRouter and lifespan support a modular monolith without requiring a large framework.

53. When Flask may be preferable

Choose Flask when...	Choose FastAPI when...
You want to learn the raw request/response mechanics with minimal framework behavior.	You want typed JSON APIs and automatic validation/docs.
The application is small and mostly synchronous.	The system has many DTOs, external async I/O, and API consumers.
You prefer manually choosing validation/DI libraries.	You want Pydantic and dependency wiring integrated.
You are building server-rendered pages with limited API surface.	The existing frontend already behaves as a separate API client.

Flask is excellent for fundamentals. For the full Ichigo backend, FastAPI reduces manual contract code and better matches the current architecture. Learning one small Flask application can still deepen your understanding of what FastAPI automates.

54. When Django may be preferable

Django becomes attractive if the system grows into a user/account platform requiring built-in admin, authentication, forms, migrations, and a conventional ORM-centered application. It is more opinionated and heavier than needed for a focused chess API, engine service, and retained static frontend.

55. When keeping Node may be the better decision

- The Node backend is already stable, tested, deployed, and the rewrite would not remove a real bottleneck.
- The team is strongest in TypeScript and values sharing generated/static types with the browser.
- Most work is lightweight real-time I/O and the Python ML system can remain a separate service.
- Operational complexity from model/engine resources inside web workers would outweigh source-code simplification.
- The migration would delay user-facing improvements without a clear learning or maintenance benefit.

56. Hybrid option: Node web server plus Python analysis service

Browser

- > Node/Express API for pages, games, realtime UI
- > HTTP/RPC request to Python analysis service
- > python-chess / ML / batch engine analysis

This preserves a mature Node server while isolating Python-native analysis. The cost is an additional network contract, deployment unit, failure mode, and data serialization boundary. For a personal project where Python is your strongest language, a single FastAPI modular monolith is usually easier initially.

Part VIII - Critical review and final recommendation

57. Critical review of the proposed Python rewrite

57.1 Strong reasons to migrate

- Your strongest language becomes the backend implementation language, reducing incidental syntax friction.
- python-chess consolidates legality, FEN, PGN, outcomes, and engine communication in one ecosystem.
- The ML scoring subprocess boundary can be removed when in-process inference is operationally acceptable.
- FastAPI makes endpoint contracts and validation more visible than ad hoc request-body checks.
- The rewrite is a valuable architecture-learning exercise because the browser/server boundary remains constant while the implementation changes.

57.2 Risks that could make the rewrite worse

Risk	Failure mode	Mitigation
Big-bang rewrite	Many endpoints fail at once; no trusted baseline.	Contract fixtures and endpoint-by-endpoint cutover.
Async cargo cult	Blocking engine/model/DB calls freeze event loop.	Choose def/async def based on library behavior; profile.
Resource duplication	Each worker loads model and engine.	Explicit worker count and separate resource workers.
DTO drift	Frontend breaks on renamed fields or changed errors.	Aliases and contract tests.
Rule behavior drift	python-chess and chess.js edge outcomes differ.	Replay corpus and terminal-state regression tests.
SQLite contention	Long transactions cause locked database errors.	Short transactions, timeout/WAL review, job separation.
Over-architecture	Too many abstractions hide simple flows.	Start minimal; extract only around real responsibilities.
In-process ML coupling	Model crash/memory pressure impacts web API.	Adapter boundary; move to worker/service when needed.

58. Corrections to common oversimplifications

Oversimplification	More accurate statement
"FastAPI replaces Node."	Python replaces the server implementation; Uvicorn replaces the HTTP server runtime role; FastAPI replaces Express framework responsibilities.
"Python can replace JavaScript."	Python replaces backend JavaScript here; browser interaction remains JavaScript unless the frontend is redesigned.
"async def is faster."	async def helps when awaiting non-blocking I/O; it can be worse when it directly calls blocking work.
"One model loaded once."	It is loaded once per worker process unless externalized or architecture-specific sharing is used.
"BackgroundTasks solves long jobs."	It performs post-response work in the web process; it is not a durable queue.
"Pydantic validates the move."	Pydantic validates the request shape/syntax; python-chess validates move legality in position context.
"Same SQLite file means zero migration risk."	File format is compatible, but schema, transactions, concurrency, and stored data assumptions still require testing.

59. Internal consistency review performed for this document

The guide was reviewed against the following criteria before delivery:

- Every proposed backend responsibility has an identified Python owner.

- Browser-only responsibilities are not incorrectly assigned to Python.
- FastAPI, Uvicorn, ASGI, Pydantic, python-chess, SQLite, Stockfish, and ML process boundaries are distinguished.
- Code examples preserve the current frontend's important field names and endpoint shapes.
- Synchronous and asynchronous examples state why each execution model is chosen.
- Engine and model lifecycle examples account for startup, serialization, shutdown, and worker multiplication.
- Long analysis is not incorrectly presented as a normal in-request or BackgroundTasks workload.
- Database examples use parameterized SQL and explicit commit/rollback behavior.
- The recommended architecture remains a modular monolith rather than adding unnecessary distributed components.
- The migration plan includes baseline capture, compatibility testing, cutover, and rollback considerations.

59.1 Known simplifications

- The code is a coherent architecture example, not a drop-in replacement for unseen portions of the current server.js.
- Exact database columns, current endpoint error bodies, opening-book storage, engine flags, and puzzle DTO variants must be verified against the repository.
- Production authentication, CSRF strategy, rate limiting, database migrations, and job-queue technology require separate implementation decisions.
- Engine concurrency behavior should be load-tested with the exact Stockfish build and python-chess version used.

60. Final recommendation

Recommendation: Use FastAPI for the Ichigo backend, keep the frontend JavaScript, preserve the current HTTP contracts, and migrate endpoint by endpoint. Begin with game creation and legal-move persistence using raw sqlite3 and python-chess. Add a lifespan-managed engine only after the core game contract passes. Move personalized multi-game analysis to a job worker when its duration or resource use makes ordinary HTTP requests unreliable.

This approach teaches the complete web-system picture rather than avoiding it. You will still learn HTTP, sockets, ports, browser/server separation, routing, middleware, request validation, persistence, concurrency, process management, and API contracts. Python simply lets you learn those concepts using a backend language you already understand well.

Appendix A - Node-to-FastAPI cheat sheet

Node / Express	Python / FastAPI	Note
node server.js	uvicorn app.main:app	Uvicorn owns the socket.
app = express()	app = FastAPI()	Framework application object.
app.get(path, handler)	@app.get(path)	Method/path registration.
express.Router()	APIRouter()	Feature routers.
app.use(router)	app.include_router(router)	Router composition.
express.json()	Pydantic/body parsing built into route declarations	Validation differs.
req.params.id	game_id: int path parameter	Typed conversion/validation.
req.query.limit	limit: int query parameter	Constraints via Query.
req.body	Pydantic model parameter	Typed DTO.
res.json(value)	return value/model	Serialization after return.

Node / Express	Python / FastAPI	Note
res.status(404)	HTTPException or exception handler	Central mapping recommended.
next()	await call_next / Depends / exception flow	No literal equivalent.
express.static()	StaticFiles mount	Mounted sub-application.
app.listen()	Uvicorn command	Server/framework roles split.
better-sqlite3	sqlite3 or SQLAlchemy	Choose sync/async model deliberately.
chess.js	python-chess	Regression-test behavior.
child_process	subprocess/asyncio/python-chess engine	Resource manager recommended.
fetch/axios server-side	httpx	Use AsyncClient for async I/O.
npm/package.json	pip/uv + pyproject.toml	Dependency ecosystem.
Jest/Vitest/Supertest	pytest/TestClient/httpx	Layered tests.

Appendix B - Complete sample project skeleton

The following compact skeleton shows how the pieces fit. It is intentionally smaller than the full recommended structure but preserves the key boundaries.

B.1 File tree

```

app/
├── __init__.py
├── main.py
├── config.py
├── api/
│   ├── dependencies.py
│   ├── exception_handlers.py
│   └── routes/
│       ├── games.py
│       └── pages.py
├── domain/
│   ├── errors.py
│   └── game.py
├── schemas/
│   └── game.py
├── services/
│   ├── chess_rules.py
│   └── game_service.py
├── repositories/
│   └── sqlite_game_repository.py
└── infrastructure/
    └── database.py

```

B.2 Minimal main.py without engine

```

from contextlib import asynccontextmanager

from fastapi import FastAPI
from fastapi.staticfiles import StaticFiles

from app.api.exception_handlers import install_exception_handlers
from app.api.routes import games, pages
from app.config import get_settings
from app.infrastructure.database import initialize_database

@asynccontextmanager
async def lifespan(app: FastAPI):
    settings = get_settings()
    initialize_database(settings.database_path)
    yield

def create_app() -> FastAPI:
    settings = get_settings()
    app = FastAPI(title="Ichigo Chess", lifespan=lifespan)
    app.include_router(games.router)
    app.include_router(pages.router)
    app.mount(
        "/static",
        StaticFiles(directory=settings.public_dir,
                    name="static",
                    )
    )
    install_exception_handlers(app)
    return app

app = create_app()

```

B.3 API contract smoke commands

```

# Health/page
curl -i http://localhost:8080/

# Create game

```

```

curl -i -X POST http://localhost:8080/game/new \
-H 'Content-Type: application/json' \
-d '{"time_control": "300+0"}'

# Submit move (replace 1 with returned gameId)
curl -i -X POST http://localhost:8080/game/1/move \
-H 'Content-Type: application/json' \
-d '{"uci": "e2e4"}'

# Illegal move example
curl -i -X POST http://localhost:8080/game/1/move \
-H 'Content-Type: application/json' \
-d '{"uci": "e2e5"}'

```

B.4 Migration definition of done

- The browser loads all HTML, CSS, JavaScript modules, board assets, and fonts from the Python-served origin.
- Every current endpoint has a documented method, path, request schema, response schema, and error behavior.
- Legal and illegal move corpora produce expected acceptance, FEN, history, results, and PGN.
- Engine requests have bounded duration, serialized access, and clean shutdown.
- Puzzle DTOs use one canonical internal schema even when external sources differ.
- The Node server is not needed for any normal Play or Puzzle flow.
- Tests and logs can distinguish browser, API, database, engine, upstream HTTP, and model failures.

Appendix C - Glossary and source references

C.1 Glossary

Term	Meaning in the Python design
ASGI	Python protocol between an async-capable server such as Uvicorn and an application such as FastAPI.
Uvicorn	Server process that binds the port, accepts connections, and invokes the ASGI app.
FastAPI	Framework that defines routes, validation, dependencies, middleware, OpenAPI, and response handling.
Path operation	FastAPI term for a method/path handler such as POST /game/{game_id}/move.
Pydantic model	Typed request/response data model with parsing and validation.
Dependency	A callable resolved by FastAPI to provide a connection, user, service, adapter, or other request input.
Lifespan	Startup/shutdown context around the application process.
Repository	Abstraction that hides persistence operations from application services.
Application service	Object/function that orchestrates a complete use case.
UCI	Coordinate move protocol and engine communication standard.
FEN	Snapshot representation of board position and metadata.
PGN	Game notation format including moves and metadata.
Worker	Separate process running a copy of the application.
ThreadPool	Threads used to execute blocking synchronous work without running it directly on the event loop.
Job queue	Persistent or brokered mechanism for work that outlives an HTTP request.

C.2 Primary project source

Ichigo Chess - High-Level Architecture (HLD) and Low-Level Design (LLD), supplied 25 July 2026. This guide preserves its central browser/server boundary, server-authoritative Play model, feature-layered frontend, SQLite persistence, engine integration, and puzzle pipeline context.

C.3 Official references reviewed

- [FastAPI: Concurrency and async/await](#)
- [FastAPI: Request Body](#)
- [FastAPI: Response Model](#)
- [FastAPI: Dependencies](#)
- [FastAPI: Dependencies with yield](#)
- [FastAPI: Bigger Applications / APIRouter](#)
- [FastAPI: Middleware](#)
- [FastAPI: Lifespan Events](#)
- [FastAPI: Static Files](#)
- [FastAPI: Background Tasks](#)
- [FastAPI: Server Workers](#)
- [Express: Serving static files](#)
- [Express: Error handling / application API](#)
- [Express 5 migration guide](#)
- [Python standard library](#)
- [python-chess core documentation](#)
- [python-chess UCI/XBoard engine communication](#)
- [python-chess PGN parsing/writing](#)

C.4 Final implementation note

Before applying this guide directly, inspect the current repository's real server.js, package scripts, database schema, engine command flags, frontend gateway DTOs, and puzzle response variants. The migration should be driven by verified contracts and tests, not by assumptions from architecture prose alone.